॥वसुधैव कुटुम्बकम्॥

Symbiosis Institute of Technology, Pune

Faculty of Engineering

CSE - Academic Year 2026-27

Java Programming – Lab (Batch 2025-29)

Lab Assignment No:- 27

Name of Student	Abobakr Mohamed Elameen
PRN No.	25070122015
Batch	2025-29
Class	Programming with JAVA
Academic Year & Semester	2nd Year – Sem III
Date of Submission	03/10/2026
Title of Assignment	Develop a Java application demonstrating socket programming for communication between a client and a server.

Practice questions

1. Create a simple client-server chat application using sockets.

ChatServer.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class ChatServer {

    public static void main(String[] args) {
        try (ServerSocket serverSocket = new ServerSocket(5000)) {
            System.out.println("Server started. Waiting for
client...");
            Socket socket = serverSocket.accept();
            System.out.println("Client connected.");

            BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            PrintWriter out = new
PrintWriter(socket.getOutputStream(), true);
            Scanner sc = new Scanner(System.in);
```

```
            String msg;
            while ((msg = in.readLine()) != null) {
                System.out.println("Client: " + msg);
                if (msg.equalsIgnoreCase("bye")) {
                    break;
                }
                System.out.print("Server: ");
                String reply = sc.nextLine();
                out.println(reply);
                if (reply.equalsIgnoreCase("bye")) {
                    break;
                }
            }

            socket.close();
            System.out.println("Chat ended.");
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

ChatClient.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class ChatClient {

    public static void main(String[] args) {
        try (Socket socket = new Socket("localhost", 5000)) {
            System.out.println("Connected to server.");

            BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            PrintWriter out = new
PrintWriter(socket.getOutputStream(), true);
            Scanner sc = new Scanner(System.in);

            while (true) {
                System.out.print("Client: ");
                String msg = sc.nextLine();
                out.println(msg);
                if (msg.equalsIgnoreCase("bye")) {
                    break;
                }

                String reply = in.readLine();
```

```
                if (reply == null) {
                    break;
                }
                System.out.println("Server: " + reply);
                if (reply.equalsIgnoreCase("bye")) {
                    break;
                }
            }

            System.out.println("Chat ended.");
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

2. Create a Java program using socket programming where the client sends a message to the server, and the server returns the same message back (echo system).

EchoServer.java

```
import java.io.*;
import java.net.*;

public class EchoServer {

    public static void main(String[] args) {
        try (ServerSocket serverSocket = new ServerSocket(6000)) {
            System.out.println("Echo server started. Waiting for
client...");
            Socket socket = serverSocket.accept();
            System.out.println("Client connected.");

            BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            PrintWriter out = new
PrintWriter(socket.getOutputStream(), true);

            String msg;
            while ((msg = in.readLine()) != null) {
                System.out.println("Received: " + msg);
                if (msg.equalsIgnoreCase("exit")) {
                    break;
                }
                out.println(msg);
                System.out.println("Echoed  : " + msg);
            }
```

```
            socket.close();
            System.out.println("Echo server stopped.");
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

EchoClient.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

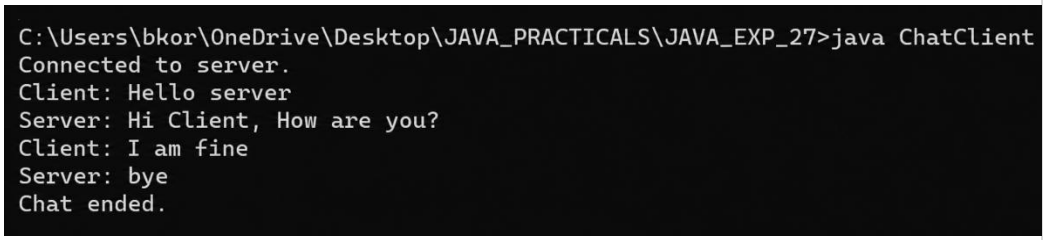
public class EchoClient {

    public static void main(String[] args) {
        try (Socket socket = new Socket("localhost", 6000)) {
            System.out.println("Connected to echo server.");

            BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            PrintWriter out = new
PrintWriter(socket.getOutputStream(), true);
            Scanner sc = new Scanner(System.in);

            while (true) {
                System.out.print("Enter message (type exit to
quit): ");
                String msg = sc.nextLine();
                out.println(msg);
                if (msg.equalsIgnoreCase("exit")) {
                    break;
                }
                String echo = in.readLine();
                if (echo == null) {
                    break;
                }
                System.out.println("Server echoed: " + echo);
            }

            System.out.println("Client closed.");
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Output	1)   2)  
Conclusion	Thus, we have successfully created and executed multiple threads in Java using the Runnable interface. The programs show tasks running concurrently, including deposit and withdrawal operations on a shared account, and how Runnable keeps task logic separate from thread execution. Thus, we have successfully developed Java applications demonstrating socket programming for communication between a client and a server. The chat application and the echo system show how data is exchanged over a network using the Socket and ServerSocket classes.